Analyze the entire project first before creating or changing anything.

Create the following "/brain/security/" documentation based strictly on the actual current project:

```
security/
├── threat-model.md
├── secrets.md
├── attack-surface.md
└── security-checklist.md
```

Requirements

- Inspect the complete codebase, architecture, frontend, backend, APIs, authentication, authorization, database, storage, dependencies, environment variables, deployment configuration, third-party services, and data flows.
- Do not modify, rewrite, refactor, move, rename, delete, or replace any existing code or configuration.
- Do not change application behavior.
- Do not invent security issues. Every finding must be based on actual project evidence.
- Never expose, copy, or print actual secret values, API keys, tokens, passwords, private keys, or credentials. Identify only their location/type and risk.
- Understand existing security mechanisms before identifying gaps.

Generate:

"threat-model.md"

- Assets
- Trust boundaries
- Actors
- Entry points
- Threats
- Attack scenarios
- Impact
- Likelihood
- Risk severity
- Existing mitigations
- Missing mitigations

"secrets.md"

- Where secrets are expected
- Environment/configuration handling
- Hardcoded-secret risks

- Client-side exposure risks
- Git/history exposure risks
- Logging/error exposure risks
- Secret rotation requirements
- Actual findings with evidence

"attack-surface.md"

- Public endpoints
- Internal endpoints
- Authentication/authorization surfaces
- User inputs
- File uploads
- Database access
- Storage
- Webhooks
- Third-party integrations
- Admin surfaces
- Network/deployment exposure
- Dependencies and other externally reachable surfaces

"security-checklist.md"
Create a project-specific checklist covering:

- Authentication
- Authorization/RBAC
- Session/token security
- Input validation
- Injection
- XSS/CSRF
- API security
- Rate limiting/abuse
- File uploads
- Database security
- Secrets
- Encryption
- CORS
- Dependency security
- Logging/monitoring
- Admin security
- Deployment/infrastructure
- Data privacy
- Error handling
- Backup/recovery

For every important finding include:

"Severity → Evidence → Risk → Affected location → Why it matters → Recommended fix"

Clearly separate:

- Confirmed vulnerability
- Security weakness
- Missing control
- Potential risk requiring verification
- Recommendation

Do not fix anything automatically.

Before finishing, verify that the documentation matches the current project implementation and does not claim controls or protections that the project does not actually have.

The final output should be the four Markdown files only, with an accurate security assessment of the existing project.